



Product Requirements Document (PRD)

Product: Orvio — Nigerian, lightweight Inventory + POS with WhatsApp Orders & Local Payments

Version: 0.9 (MVP-focused)

Date: 2026-09-28

Owner: Product (Code X / Orviahub team)

Executive Summary

Orvio is a Nigerian, multi-tenant SaaS for small retailers and wholesalers that unifies **inventory, POS, and WhatsApp orders with local payments** (Paystack/Flutterwave/Moniepoint) in one lightweight system. It is designed to be simpler and cheaper than full ERPs (e.g., Odoo) while providing real stock control, offline-capable POS, and WhatsApp-first selling that matches how Nigerian SMBs actually operate. ^{[1] [2] [3] [4] [5] [6] [7] [8]}

The MVP focuses on two core modules—**Orvio Inventory** and **Orvio POS**—plus a **WhatsApp Orders** channel and a **Payments** abstraction layer. The system is multi-tenant from day one, with organizations, branches, products, stock movements, sales orders, and payment intents as core entities. ^{[2] [3] [9] [10]}

1. Product Vision

Vision

Enable Nigerian small businesses to **sell everywhere and track everything** with a simple, affordable, and reliable operations platform.

Mission

Provide a lightweight, Nigeria-native inventory and POS system that works offline, integrates WhatsApp selling, and accepts local payments without ERP complexity. ^{[7] [11] [2]}

Problem Statement

- Nigerian SMBs lose sales and profit visibility due to **stockouts, theft, and manual spreadsheets**. ^{[6] [1]}
- Existing tools are either **too heavy/expensive** (Odoo, SAP) or **too narrow** (payment-only POS, basic invoicing). ^{[5] [12] [2]}

- Internet/power unreliability makes cloud-only POS risky; many shops need offline-first behavior. ^[11] ^[13] ^[14]
- WhatsApp is a primary sales channel, but most tools don't sync WhatsApp orders to inventory and payments automatically. ^[8] ^[10] ^[7]

Opportunity Statement

- Nigeria's inventory software market is growing from ~USD 75M (2025) to ~USD 168M (2031). ^[15] ^[1]
- SMBs want affordable, Naira-priced, locally supported tools with clear ROI. ^[16] ^[2] ^[6]
- A focused product can win by being lighter than ERPs and deeper on inventory than payment apps. ^[9] ^[5]

Value Proposition

- **For shop owners:** See real stock, avoid stockouts, know daily profit, and sell via POS + WhatsApp with automatic stock updates. ^[2] ^[6] ^[7]
- **For staff:** Fast checkout, simple stock adjustments, clear roles and PINs. ^[9] ^[16]
- **For accountants/owners:** Clean sales, tax, and cost data; easy CSV exports; payment reconciliation. ^[10] ^[17]

Target Market

- **Primary:** Small retailers, mini-marts, provision stores, pharmacies, fashion boutiques, spare-parts shops in urban/semi-urban Nigeria. ^[1] ^[6] ^[2]
- **Secondary:** Small wholesalers/distributors with 1–5 branches.

Target Users

- Business owners (admins)
- Store managers
- Cashiers/sales staff
- Storekeepers/warehouse staff
- Accountants/bookkeepers (read-only/reporting)

Product Principles

1. **Nigeria-first:** Naira, VAT/WHT concepts, local receipts, WhatsApp & payment rails. ^[3] ^[7] ^[2]
2. **Lightweight by default:** Only what SMBs need to run daily ops.
3. **Offline-capable:** POS must keep selling when internet/power fails. ^[13] ^[14] ^[11]
4. **Unified stock:** All channels (POS, WhatsApp, manual) update the same inventory ledger.
5. **Transparent pricing:** Simple plans, no module nickel-and-diming. ^[16]
6. **Secure & compliant:** NDPA-aware, role-based access, audit logs. ^[18] ^[10]

Differentiators

- WhatsApp orders + payments + inventory in one flow, not separate tools. [\[7\]](#) [\[8\]](#) [\[10\]](#)
- Offline-first POS with robust sync and conflict rules. [\[14\]](#) [\[11\]](#) [\[13\]](#)
- Nigerian SMB pricing & UX (Naira, simple onboarding, WhatsApp support). [\[6\]](#) [\[2\]](#) [\[16\]](#)

Positioning

“Orvio is the lightweight, Nigerian inventory and POS platform for shops that sell in-store and on WhatsApp.”

Objectives

- **Short-term (0–6 months):** Launch MVP (Inventory + POS + WhatsApp Orders + Payments) with 50–100 paying shops in Lagos. [\[1\]](#) [\[2\]](#)
- **Medium-term (6–18 months):** Add purchasing, advanced reports, multi-branch optimizations, and more payment providers.
- **Long-term (18+ months):** Expand to eCommerce, light manufacturing, and integrations (accounting, logistics).

2. Problem Statement

See “Problem Statement” under Product Vision; this PRD treats those points as validated problem hypotheses to be tested with early users. [\[12\]](#) [\[6\]](#) [\[1\]](#)

3. Goals & Objectives

Business Goals

- Acquire 100 paying SMBs within 6 months of launch in Lagos. [\[2\]](#) [\[1\]](#)
- Achieve >80% 90-day retention among onboarded shops.
- Maintain gross margin >70% on SaaS subscriptions.

Product Goals

- Reduce stockouts and shrinkage by giving real-time stock visibility and controlled adjustments. [\[6\]](#) [\[2\]](#)
- Enable at least 30% of early merchants to take ≥10% of orders via WhatsApp within 3 months. [\[8\]](#) [\[7\]](#)
- Ensure POS remains usable during typical internet outages (offline-first). [\[11\]](#) [\[13\]](#) [\[14\]](#)

User Goals

- Know daily sales and gross profit without manual calculation. ^[16] ^[2]
- Avoid selling out-of-stock items.
- Onboard products quickly via CSV and start selling within 1 day. ^[6]

4. Target Users

Primary:

- Shop Owner (Admin)
- Store Manager
- Cashier / Sales Staff
- Storekeeper / Warehouse Staff

Secondary:

- Accountant / Bookkeeper (read-only, reports, exports)
- Support Agent (Orvio internal) (admin tools, impersonation with consent)

5. User Personas

Persona 1: “Owner-Admin” (Tunde)

- **Role:** Business owner, final decision maker.
- **Goals:** Know profit, control stock, prevent theft, keep costs low. ^[2] ^[16]
- **Motivations:** Grow business, reduce losses, have reliable records for tax/accountant.
- **Pain points:** Spreadsheets, staff mistakes, stockouts, no clear profit view.
- **Tech proficiency:** Moderate (smartphone, WhatsApp, basic apps).
- **Usage frequency:** Daily dashboards, weekly reports.
- **Key permissions:** Full org admin, billing, roles, branch management.
- **Must not:** Be locked out by staff; must retain override controls.

Persona 2: “Store Manager” (Aisha)

- **Role:** Runs day-to-day operations for 1–3 branches.
- **Goals:** Smooth sales, accurate stock, trained staff.
- **Pain points:** Staff errors, reconciliation headaches, internet outages.
- **Usage:** Daily stock checks, adjustments, reports, staff management.
- **Permissions:** Manage branch staff, approve adjustments, view reports.
- **Must not:** Change billing/plan; cannot delete audit logs.

Persona 3: “Cashier” (Emeka)

- **Role:** Operates POS, handles payments and receipts.
- **Goals:** Fast checkout, clear till reconciliation.
- **Pain points:** Slow systems, network failures, complex UIs.
- **Usage:** High-frequency POS sessions.
- **Permissions:** Create sales, apply allowed discounts, void with reason (if permitted).
- **Must not:** Change product costs, view full profit reports, delete transactions.

Persona 4: “Storekeeper” (Bisi)

- **Role:** Manages stock, receives goods, does counts.
- **Goals:** Accurate stock, fewer discrepancies.
- **Pain points:** Manual logs, unclear adjustments, theft.
- **Usage:** Stock movements, adjustments, receiving.
- **Permissions:** Create stock movements/adjustments (with reason), view stock.
- **Must not:** Approve high-value adjustments without manager.

Persona 5: “Accountant” (Mr. Okon)

- **Role:** External or internal accountant.
- **Goals:** Clean data for tax and accounts.
- **Pain points:** Messy records, missing receipts.
- **Usage:** Weekly/monthly reports, exports.
- **Permissions:** Read-only access to sales, stock, payments; export.
- **Must not:** Modify transactions or stock.

6. Jobs-to-be-Done (JTBD)

- When a shop owner opens the app in the morning, they want to see yesterday’s sales, gross profit, and low-stock items so that they can make quick restocking and pricing decisions. [\[16\]](#) [\[2\]](#)
- When a cashier starts a shift, they want a fast POS that works even if internet drops so that they don’t lose sales. [\[13\]](#) [\[11\]](#)
- When a customer sends a cart on WhatsApp, the merchant wants the order to auto-create in Orvio with a payment link so that stock and payments are tracked without manual work. [\[10\]](#) [\[7\]](#) [\[8\]](#)
- When stock is received or damaged, the storekeeper wants to record it with a reason so that inventory stays accurate and shrinkage is visible. [\[2\]](#) [\[6\]](#)

7. User Journeys

7.1 Discovery → Sign-up → Onboarding

- **Entry point:** Landing page, referral link, WhatsApp ad.
- **Intent:** “Try a simple inventory/POS for my shop.”
- **Steps:**
 1. User visits site → clicks “Start Free”.
 2. Registers with email/phone + password (or OAuth later).
 3. Verifies email/phone.
 4. Creates Organization (business name, timezone, currency=NGN).
 5. Creates first Branch (store/warehouse).
 6. Imports products via CSV or adds manually.
 7. Invites staff (optional).
 8. Starts using Inventory and/or POS.
- **Validation:** Email/phone verification, password strength, CAC/TIN optional fields.
- **Error states:** Duplicate email, invalid CSV, failed invite.
- **Success state:** Dashboard with stock summary and “Create Sale” CTA.
- **Notifications:** Welcome email, onboarding tips, invite emails.
- **Exit state:** User on dashboard with sample or real data.

7.2 Authentication (Login/Logout)

- **Entry:** Login page.
- **Intent:** Access Orvio securely.
- **Flow:**
 1. Enter email/phone + password.
 2. System validates credentials, issues session + refresh token.
 3. Redirect to dashboard.
- **Error states:** Invalid credentials, locked account, expired session.
- **Notifications:** Suspicious login email (future).
- **Exit:** Logged-in dashboard or login error message.

7.3 Initial Setup (Org/Branch/Products)

- **Intent:** Configure org for real use.
- **Steps:**
 1. Create Org (name, address, currency NGN, tax settings).

2. Create Branch (name, type: store/warehouse, address).
 3. Add Products (SKU, name, prices, tax, barcode).
 4. Set initial stock per branch (via CSV or UI).
- **Validation:** Unique SKU per org, non-negative initial stock.
 - **Error states:** Duplicate SKU, invalid CSV format.
 - **Success state:** Products listed with stock > 0.
 - **Notifications:** “Import completed” email/in-app.

7.4 Core Product Usage – Inventory

- **Intent:** Track and control stock.
- **Steps:**
 1. View stock by branch.
 2. Create stock adjustment (reason: damage, loss, found).
 3. Transfer stock between branches.
 4. View low-stock alerts and reports.
- **Validation:** Reason required, negative stock not allowed.
- **Error states:** Insufficient stock for transfer/adjustment.
- **Success state:** Updated stock, audit log entry.
- **Notifications:** Low-stock alerts (in-app, optional email).

7.5 Core Product Usage – POS

- **Intent:** Sell quickly in-store.
- **Steps:**
 1. Open POS session at branch.
 2. Scan/add items, apply discount (if allowed).
 3. Select payment method (cash, transfer, card).
 4. Complete sale, print/send receipt.
 5. End-of-day reconciliation (cash vs system).
- **Validation:** Stock available, discount within role limits.
- **Error states:** Out-of-stock item, payment failure.
- **Success state:** Sale recorded, stock reduced, receipt issued.
- **Notifications:** End-of-day summary to manager/owner.

7.6 WhatsApp Orders

- **Intent:** Capture orders from WhatsApp and collect payment.
- **Steps:**
 1. Customer browses catalog in WhatsApp, sends cart.
 2. Orvio receives order (pending_payment).
 3. System sends payment link via WhatsApp.
 4. Customer pays; webhook marks order paid.
 5. Stock reduced, receipt sent via WhatsApp.
- **Validation:** Stock check, payment link generation.
- **Error states:** Payment failed, stock insufficient.
- **Success state:** Order fulfilled, stock updated.
- **Notifications:** Order created, payment received, delivery updates. [\[19\]](#) [\[7\]](#) [\[10\]](#)

7.7 Payments & Subscription

- **Intent:** Subscribe to Orvio plan.
- **Steps:**
 1. Choose plan (Free/Starter/Growth).
 2. Enter payment details via Paystack/Flutterwave.
 3. Payment success → activate plan.
 4. Webhook handles renewal/failure.
- **Validation:** Card/transfer success, plan limits.
- **Error states:** Payment failure, card declined.
- **Success state:** Active subscription, access granted.
- **Notifications:** Receipt, renewal reminders, failure alerts. [\[20\]](#) [\[21\]](#) [\[10\]](#)

7.8 Settings, Notifications, Support

- **Intent:** Configure org, manage users, get help.
- **Steps:**
 1. Update org/branch settings.
 2. Manage users/roles.
 3. Configure notifications (email, WhatsApp templates).
 4. Contact support (in-app, WhatsApp, email).
- **Validation:** Role permissions, NDPA consent flags.
- **Error states:** Permission denied, invalid settings.
- **Success state:** Settings saved, support ticket created.

7.9 Account Recovery & Deletion

- **Recovery:** Email/phone reset flow, MFA (future).
- **Deletion:** Owner requests → soft delete → retention period → hard delete (per policy/NDPA).
[\[18\]](#) [\[10\]](#)

7.10 Upgrades/Downgrades

- **Upgrade:** Immediate feature access, proration (future).
- **Downgrade:** Apply at period end, enforce limits gracefully.

7.11 Administrative Workflows

- **Internal admin:** Suspend org, reset MFA, view audit logs, impersonate (with consent & logging).

8. Product Scope

In Scope (MVP / Phase 1)

- Multi-tenant org/branch model.
- Inventory: products, stock, adjustments, transfers, low-stock alerts, CSV import/export. [\[6\]](#)
[\[2\]](#)
- POS: offline-capable checkout, receipts, basic reports, staff PINs. [\[9\]](#) [\[11\]](#) [\[16\]](#)
- WhatsApp Orders: catalog sync, order capture, payment links, status updates. [\[19\]](#) [\[7\]](#) [\[10\]](#)
- Payments abstraction: Paystack (primary), Flutterwave/Moniepoint (secondary), webhooks. [\[4\]](#) [\[21\]](#) [\[3\]](#) [\[20\]](#)
- Roles & permissions (owner, manager, cashier, storekeeper, accountant).
- Basic reports: sales, stock, gross profit.
- NDPA-aware consent and data export/delete (basic). [\[10\]](#) [\[18\]](#)

Phase 2

- Purchasing (POs, GRN).
- Advanced reports (cohort, ABC analysis).
- Multi-branch optimizations (centralized purchasing, replenishment).
- More payment providers, virtual accounts.

Phase 3+

- eCommerce site builder.
- Light manufacturing/BOM.
- Accounting integrations (Sage, QuickBooks).
- Logistics/delivery integrations.

9. Functional Requirements

(Illustrative subset with IDs; full implementation should expand similarly.)

Module: AUTH (Authentication & Sessions)

AUTH-001 – Registration

- **Actor:** Prospective user.
- **Trigger:** Submit sign-up form.
- **Behavior:** Create user, send verification email/SMS.
- **Rules:** Unique email/phone, password policy.
- **Success:** User created, verification sent.
- **Failure:** Error message, no account created.
- **Audit:** User registration event.

AUTH-002 – Login

- **Actor:** Registered user.
- **Trigger:** Submit credentials.
- **Behavior:** Validate, issue session + refresh token.
- **Rules:** Rate limiting, brute-force protection.
- **Success:** Redirect to dashboard.
- **Failure:** Generic error, increment failure count.
- **Audit:** Login attempt logged.

AUTH-003 – Logout

- **Actor:** Logged-in user.
- **Trigger:** Click logout.
- **Behavior:** Invalidate session.
- **Success:** Redirect to login.
- **Audit:** Logout event.

AUTH-004 – Password Reset

- **Actor:** User.
- **Trigger:** Request reset.
- **Behavior:** Send reset link, allow new password.
- **Rules:** Link expiry, one-time use.
- **Audit:** Reset requested/completed.

Module: ORG (Organization & Branch)

ORG-001 – Create Organization

- **Actor:** Owner.
- **Trigger:** Onboarding flow.
- **Behavior:** Create org with NGN, timezone.
- **Rules:** One org per signup (MVP).
- **Success:** Org created, user as owner.
- **Audit:** Org creation.

ORG-002 – Create Branch

- **Actor:** Owner/Manager.
- **Trigger:** Add branch.
- **Behavior:** Create branch under org.
- **Rules:** Unique branch name per org.
- **Success:** Branch listed.
- **Audit:** Branch creation.

Module: INV (Inventory)

INV-001 – Create Product

- **Actor:** Owner/Manager/Storekeeper.
- **Trigger:** Add product.
- **Behavior:** Create product (SKU, name, prices, tax, barcode).
- **Rules:** Unique SKU per org, non-negative prices.
- **Success:** Product saved.
- **Failure:** Duplicate SKU error.
- **Audit:** Product create.

INV-002 – Set Initial Stock

- **Actor:** Storekeeper/Manager.
- **Trigger:** Import CSV or UI entry.

- **Behavior:** Create stock items per branch.
- **Rules:** Non-negative quantity.
- **Success:** Stock updated.
- **Audit:** Stock initialization.

INV-003 – Stock Adjustment

- **Actor:** Storekeeper/Manager.
- **Trigger:** Record damage/loss/found.
- **Behavior:** Create StockMovement with reason.
- **Rules:** Reason required, cannot go negative.
- **Success:** Stock adjusted, audit logged.
- **Failure:** Insufficient stock error.
- **Audit:** Adjustment event.

INV-004 – Stock Transfer

- **Actor:** Manager/Storekeeper.
- **Trigger:** Transfer between branches.
- **Behavior:** Create outbound + inbound movements.
- **Rules:** Source must have stock.
- **Success:** Stock moved.
- **Audit:** Transfer event.

INV-005 – Low-Stock Alert

- **Actor:** System.
- **Trigger:** Stock < threshold.
- **Behavior:** Generate alert.
- **Rules:** Threshold per product or global.
- **Notifications:** In-app, optional email.

Module: POS (Point of Sale)

POS-001 – Create Sale

- **Actor:** Cashier.
- **Trigger:** Complete checkout.
- **Behavior:** Create SalesOrder, reduce stock.
- **Rules:** Stock must exist, discount within role limit.
- **Success:** Sale recorded, receipt issued.
- **Failure:** Out-of-stock, discount exceeded.

- **Audit:** Sale event.

POS-002 – Void Sale

- **Actor:** Manager/Cashier (if allowed).
- **Trigger:** Void request with reason.
- **Behavior:** Reverse stock, mark sale void.
- **Rules:** Reason required, permission check.
- **Audit:** Void event.

POS-003 – End-of-Day Reconciliation

- **Actor:** Manager/Cashier.
- **Trigger:** Close shift.
- **Behavior:** Summarize cash vs system, record variance.
- **Success:** Shift closed, report saved.
- **Audit:** Shift close.

Module: WA (WhatsApp Orders)

WA-001 – Sync Catalog to WhatsApp

- **Actor:** Owner/Manager.
- **Trigger:** Enable WhatsApp channel.
- **Behavior:** Push products to WhatsApp catalog or prepare feed.
- **Rules:** Only active products, image URL required.
- **Success:** Catalog visible in WhatsApp.
- **Audit:** Catalog sync.

WA-002 – Receive WhatsApp Order

- **Actor:** System (via WhatsApp API).
- **Trigger:** Customer sends cart.
- **Behavior:** Create WhatsAppOrder (pending_payment).
- **Rules:** Validate items, calculate total.
- **Success:** Order created, payment link prepared.
- **Audit:** Order received.

WA-003 – Send Payment Link

- **Actor:** System.
- **Trigger:** Order created.
- **Behavior:** Generate Paystack/Flutterwave link, send via WhatsApp.
- **Rules:** Unique reference per order. ^[10]

- **Success:** Link sent.
- **Failure:** Retry logic, alert merchant.

WA-004 – Payment Webhook → Order Confirmed

- **Actor:** System (payment gateway).
- **Trigger:** charge.success webhook.
- **Behavior:** Mark order paid, reduce stock, send receipt.
- **Rules:** Verify signature, idempotent handling. ^[21] ^[22] ^[23]
- **Success:** Order fulfilled.
- **Audit:** Payment confirmed.

Module: PAY (Payments & Billing)

PAY-001 – Create Payment Link

- **Actor:** System.
- **Trigger:** Order or invoice.
- **Behavior:** Call Paystack/Flutterwave API, store link.
- **Rules:** NGN, metadata includes order_id.
- **Success:** Link stored.
- **Audit:** Link created.

PAY-002 – Handle Payment Webhook

- **Actor:** System.
- **Trigger:** Gateway webhook.
- **Behavior:** Verify signature, update PaymentIntent, notify.
- **Rules:** Idempotent, return 200 immediately. ^[23] ^[24] ^[21]
- **Audit:** Webhook processed.

PAY-003 – Subscription Billing

- **Actor:** System.
- **Trigger:** Renewal date.
- **Behavior:** Charge card/issue payment link, update plan.
- **Rules:** Grace period on failure.
- **Audit:** Billing event.

(Additional modules: USER, ROLE, REPORT, ADMIN follow similar patterns.)

10. Business Rules

- **Multi-tenancy:** All data scoped to `organization_id`; no cross-org access.
- **Branch scope:** Users can be restricted to specific branches.
- **Stock:** Cannot go negative; adjustments require reason.
- **Sales:** Cannot sell without stock (MVP).
- **Discounts:** Role-based maximum % or amount.
- **Voids:** Require reason and permission; audited.
- **Payments:** Webhooks must be signature-verified; idempotent processing. ^[22] ^[21] ^[23]
- **NDPA:** Consent recorded for marketing; separate consent for order comms. ^[18] ^[10]
- **Retention:** Transaction records retained ≥ 5 years for CBN compliance. ^[10]
- **Deletion:** Soft delete with retention window; hard delete per policy/NDPA request. ^[18]

11. Roles & Permissions

Roles

- **Owner:** Full org access, billing, settings, all branches.
- **Manager:** Branch management, staff, approvals, reports.
- **Cashier:** POS sales, limited discounts, voids (if allowed).
- **Storekeeper:** Stock movements, adjustments (within limits).
- **Accountant:** Read-only sales/stock/payments, exports.
- **Internal Admin (Orvio):** Org suspension, audit access, impersonation (logged).

Permission Matrix (excerpt)

Module	Owner	Manager	Cashier	Storekeeper	Accountant
View Stock	Yes	Yes	Yes	Yes	Yes
Adjust Stock	Yes	Yes	No	Yes (limit)	No
Create Sale (POS)	Yes	Yes	Yes	No	No
Void Sale	Yes	Yes	Optional	No	No
View Sales Reports	Yes	Yes	Limited	Limited	Yes
Manage Users/Roles	Yes	Branch	No	No	No
Billing/Plan	Yes	No	No	No	No
WhatsApp Settings	Yes	Yes	No	No	No
Export Data	Yes	Yes	No	No	Yes

- **Resource-level permissions:** Users can be scoped to specific branches.

- **Audit:** All permission changes logged.

12. Data Requirements

Core Entities (conceptual)

- **Organization**
 - Fields: id, name, address, timezone, currency, tax_settings, status, created_at.
 - Relationships: many Branch, many UserMembership, many Product.
- **Branch**
 - Fields: id, org_id, name, type (store/warehouse), address, status.
 - Relationships: many StockItem, many SalesOrder, many StockMovement.
- **User**
 - Fields: id, email, phone, password_hash, status, mfa_enabled.
- **UserMembership**
 - Fields: id, org_id, user_id, role, status, invited_at, accepted_at.
- **Product**
 - Fields: id, org_id, sku, name, description, cost_price, sell_price, tax_rate, barcode, category, is_active.
 - Relationships: many StockItem, many SalesOrderItem.
- **StockItem**
 - Fields: id, product_id, branch_id, quantity_on_hand, reserved_quantity.
- **StockMovement**
 - Fields: id, product_id, branch_id, type (in/out/transfer/adjust), quantity, reason, reference_type, reference_id, created_by, created_at.
- **SalesOrder**
 - Fields: id, org_id, branch_id, channel (pos/whatsapp/web), status, subtotal, tax, total, payment_status, created_by, created_at.
- **SalesOrderItem**
 - Fields: id, order_id, product_id, quantity, unit_price, discount, tax.
- **PaymentIntent / Charge**
 - Fields: id, org_id, order_id, provider, provider_ref, amount, currency, status, metadata, created_at.
- **WhatsAppChannel**
 - Fields: id, org_id, phone_number, provider, status.
- **WhatsAppOrder**

- Fields: id, org_id, customer_phone, items_json, total, status, payment_link_id, created_at.
- **Subscription**
 - Fields: id, org_id, plan, status, current_period_start, current_period_end, provider, provider_customer_id.
- **AuditLog**
 - Fields: id, org_id, actor_user_id, action, resource_type, resource_id, before_json, after_json, ip, user_agent, created_at.

(Relationships, ownership, and lifecycle rules follow the business rules above.)

13. Workflows & State Machines

SalesOrder States

Current State	Action	Required Permission	New State	Side Effects
draft	submit	Cashier/Manager	pending_payment	Reserve stock (optional)
pending_payment	payment success (webhook)	System	paid	Reduce stock, trigger fulfillment
pending_payment	payment failed	System	payment_failed	Notify customer/merchant
paid	fulfill	Manager/Storekeeper	fulfilled	Mark delivered/completed
paid	refund	Owner/Manager	refunded	Reverse stock (policy), audit
any	cancel (pre-fulfillment)	Owner/Manager	cancelled	Release reserved stock

PaymentIntent States

Current State	Action	New State	Side Effects
created	customer pays	succeeded	Update order, send notifications
created	payment fails	failed	Notify, allow retry
succeeded	refund	refunded	Reverse money, audit

Subscription States

- trial → active → past_due → canceled / reactivated.

14. UI/UX Requirements

Global

- **Navigation:** Left sidebar (Inventory, POS, Orders, Reports, Settings).
- **Responsive:** Desktop-first, tablet-friendly; POS optimized for touch.
- **Accessibility:** Keyboard navigable, sufficient contrast, ARIA labels.

Loading States

- **Skeleton:** Dashboards, lists (products, orders).
- **Spinner:** Form submissions, payment link generation.
- **Button loading:** Inline during save/pay.
- **Optimistic updates:** POS cart additions (with rollback on sync failure). ^[11] ^[13]

Empty States

- **No products:** “Add your first product” + CSV template.
- **No sales:** “Create your first sale” CTA.
- **No WhatsApp orders:** Explain how to enable WhatsApp channel.

Error States

- **User-facing:** Clear message + retry CTA.
- **Logging:** Structured logs with correlation IDs.
- **Retry:** Exponential backoff for transient failures.

Key Screens

- **Dashboard:** KPIs (sales, profit, low stock).
- **Inventory List:** Search, filters (category, branch, low stock), sort, pagination.
- **Product Form:** SKU, name, prices, tax, barcode, category.
- **POS:** Cart, item search/scan, payment selection, receipt preview.
- **Orders List:** Filter by channel, status, date.
- **Reports:** Sales by channel, stock valuation, gross profit.
- **Settings:** Org, branch, users/roles, WhatsApp, payments, billing.

15. Notifications & Events

Business Events

- `user.registered`
- `org.created`
- `product.created`
- `stock.adjusted`
- `sale.created`
- `sale.voided`
- `order.created` (WhatsApp)
- `payment.succeeded`
- `payment.failed`
- `subscription.renewed`
- `subscription.payment_failed`

Notification Rules

- **Order created (WhatsApp):** Send payment link message. ^[7] ^[10]
- **Payment succeeded:** Send receipt via WhatsApp + in-app. ^[19]
- **Low stock:** In-app alert; optional email to manager/owner.
- **Subscription failure:** Email + in-app alert to owner.

Channels

- In-app, email, WhatsApp (for customer-facing order comms), webhooks (for integrations).

16. Search / Filtering / Reporting

- **Search:** Products (by name, SKU, barcode), Orders (by ID, customer phone), Customers (by phone/name).
- **Filters:** Branch, date range, channel, status, category.
- **Sorting:** Date, total, product name, stock level.
- **Pagination:** Cursor or offset-based; configurable page size.
- **Exports:** CSV for products, sales, stock, payments.
- **Reports:**
 - Daily/weekly/monthly sales.
 - Gross profit (sales – cost).
 - Stock valuation.

- Sales by channel (POS vs WhatsApp). ^[16] ^[2]

17. Administration

Org Administration

- Manage branches, users, roles.
- Configure tax, currency (NGN locked in MVP), receipt templates.

System Administration (Orvio internal)

- View orgs, suspend/activate.
- Audit log viewer.
- Impersonation (with explicit consent flag & audit).

Support Tools

- View org health (errors, webhook failures).
- Resend invites, reset MFA.

18. Authentication & Security

- **Registration/Login:** Email/phone + password; future OAuth.
- **Sessions:** JWT or session tokens + refresh tokens; device management (future).
- **Password reset:** Token-based, expiry, one-time use.
- **MFA:** Future (TOTP/SMS).
- **Rate limiting:** On login, password reset, payment webhooks.
- **Brute-force protection:** Lockout after N failures, cooldown.
- **Audit logging:** All auth events logged.
- **Data protection:** HTTPS everywhere, secrets in vault, PII minimization.
- **NDPA compliance:** Consent capture, purpose limitation, data subject rights (export/delete).

^[18] ^[10]

19. API / Backend Requirements

Resources (illustrative)

- `POST /auth/register`, `POST /auth/login`, `POST /auth/logout`, `POST /auth/reset-password`
- `POST /orgs`, `GET /orgs/:id`, `POST /orgs/:id/branches`
- `GET/POST /products`, `POST /products/import`, `GET /products/:id/stock`

- POST /stock/adjustments, POST /stock/transfers
- POST /pos/sales, POST /pos/sales/:id/void
- POST /whatsapp/orders, GET /whatsapp/orders/:id
- POST /payments/links, POST /payments/webhooks/:provider
- GET /reports/sales, GET /reports/stock

Each endpoint must define:

- Request schema, response schema, error codes.
- Authorization (role/permission).
- Idempotency keys for writes (sales, payments).
- Pagination/filtering/sorting for lists.
- Rate limits per org/user.

20. Non-Functional Requirements

- **Performance:**
 - Dashboard load < 2s for typical org.
 - POS cart actions < 200ms local, sync < 2s when online. [\[13\]](#) [\[11\]](#)
- **Scalability:**
 - Support 1,000 orgs, 10,000 daily active users in Year 1.
 - Horizontal scaling for API and workers.
- **Availability:**
 - Target 99.5% uptime MVP; graceful degradation when payments/WhatsApp down.
- **Reliability:**
 - Idempotent webhooks, retry queues, dead-letter queues. [\[24\]](#) [\[21\]](#) [\[23\]](#)
- **Security:**
 - OWASP Top 10 mitigations, HMAC webhook verification. [\[22\]](#)
- **Accessibility:**
 - WCAG 2.1 AA target for core flows.
- **Compatibility:**
 - Chrome, Edge, Safari; Android POS tablets; low-end Windows PCs. [\[25\]](#)
- **Maintainability:**
 - Modular codebase, clear domain boundaries (Inventory, POS, Payments, WhatsApp).
- **Observability:**
 - Structured logs, metrics (latency, error rates), tracing.

21. Integrations

- **WhatsApp Business API** (via BSP: WATI/Twilio/MessageBird or Nigerian BSPs like Prembly/KwikChat). ^[26] ^[27] ^[3]
 - Data: product catalog, orders, messages.
 - Flow: Outbound catalog sync, inbound order messages, outbound status updates.
 - Failure: Queue messages, alert merchant.
- **Paystack** (primary), **Flutterwave**, **Moniepoint/Monnify**. ^[28] ^[3] ^[4] ^[20]
 - Data: payment intents, charges, webhooks.
 - Flow: Create payment link, handle webhooks, verify signatures.
 - Security: HMAC verification, HTTPS, replay protection. ^[23] ^[22]
- **Email (future)**: Transactional emails (welcome, receipts, alerts).

22. Payments & Billing

Pricing Model (to be finalized; indicative)

- **Free**: 1 branch, 1 user, limited transactions, basic inventory + POS.
- **Starter**: ~₦5k–₦8k/month, 1–2 branches, more users/transactions, basic reports.
- **Growth**: ~₦15k–₦25k/month, multi-branch, advanced reports, API, priority support. ^[2] ^[16]

Subscription Lifecycle

- Trial (optional) → Active → Past Due → Canceled.
- Webhooks handle renewal success/failure. ^[29] ^[21]

Payment Rules

- NGN only (MVP).
- Payment links include metadata (order_id, org_id). ^[10]
- Failed payments → grace period → feature restriction (read-only).

23. Analytics & Metrics

- **Business KPIs**: MRR, churn, CAC, LTV.
- **Product KPIs**:
 - Activation rate (orgs with ≥1 product & ≥1 sale in 7 days).
 - 30/90-day retention.
 - % orgs using WhatsApp orders.
 - POS offline session rate.

- **Operational Metrics:**
 - Webhook success rate.
 - Sync conflict rate.
 - Average response time.

24. Audit Logging

Auditable actions (non-exhaustive):

- User login/logout, password reset.
- Org/branch create/update/delete.
- Product create/update/delete.
- Stock adjustments, transfers.
- Sales create/void/refund.
- Payment link creation, webhook processing.
- Role/permission changes.
- Subscription changes.

Each audit record includes: actor, action, resource, before/after, timestamp, IP/device.

25. Edge Cases

- **Duplicate SKU:** Reject, suggest correction.
- **Concurrent adjustments:** Use transactions; last-write-wins with audit.
- **Network failure during POS sale:** Queue locally, sync later; idempotent APIs. ^[14] ^[11] ^[13]
- **Expired session:** Prompt re-login, preserve draft data.
- **Deleted user:** Reassign ownership, preserve audit.
- **Suspended org:** Read-only access, export allowed.
- **Payment webhook replay:** Idempotent handling via provider_ref. ^[21] ^[23]
- **Multiple tabs:** Conflict resolution on save; show “data changed” warning.
- **Offline/intermittent:** Local queue, retry with backoff. ^[14] ^[13]
- **Large datasets:** Server-side pagination, async exports.

26. Security Threat Analysis

- **Unauthorized access:** Strong auth, session management, RBAC.
- **Privilege escalation:** Strict permission checks, server-side enforcement.
- **Tenant data leakage:** org_id scoping, row-level security.
- **Session theft:** Secure cookies, short-lived tokens, rotation.

- **CSRF/XSS/Injection:** Standard web security practices, parameterized queries.
- **Broken access control:** Test all endpoints for org/branch scope.
- **Webhook spoofing:** HMAC signature verification, allowlist IPs (if provided).^[22] ^[23]
- **Sensitive data exposure:** Minimize PII, encrypt at rest/in transit.

27. MVP / Phase Planning

MVP (Phase 1)

- Org/Branch, Products, Stock, Adjustments, Transfers.
- POS (offline-capable), basic reports.
- WhatsApp Orders (catalog + payment links + webhooks).
- Paystack integration (primary), basic subscription billing.
- Roles: Owner, Manager, Cashier, Storekeeper, Accountant.
- NDPA basics (consent, export/delete).^[18] ^[10]

Phase 2

- Purchasing (PO/GRN).
- Advanced reports, ABC analysis.
- More payment providers, virtual accounts.
- Enhanced WhatsApp automation (templates, reminders).

Phase 3+

- eCommerce, logistics, accounting integrations.
- Light manufacturing, multi-currency (if needed).

28. Feature Dependencies

- **Auth → Org → Branch → Users/Roles → Inventory → POS → WhatsApp → Payments.**
- **Payments depends on Orders (sales/WhatsApp).**
- **WhatsApp Orders depends on Products and Payments.**
- **Reports depend on Sales and Stock data.**

29. Acceptance Criteria (samples)

AC-INV-001 – Create Product

- **Given** an active org,
- **When** a user with permission submits a valid product form (unique SKU, non-negative prices),
- **Then** the product is created, appears in the product list, and an audit log entry exists.

AC-POS-001 – Create Sale

- **Given** sufficient stock for items in the cart,
- **When** a cashier completes checkout with a valid payment method,
- **Then** a SalesOrder is created with status paid, stock is reduced, and a receipt is generated.

AC-WA-001 – WhatsApp Order → Payment

- **Given** a WhatsApp order in pending_payment,
- **When** the payment gateway sends a verified charge.success webhook with matching order_id,
- **Then** the order status becomes paid, stock is reduced, and a receipt message is sent via WhatsApp. ^[21] ^[7] ^[10]

AC-AUTH-001 – Login

- **Given** a verified user,
- **When** they submit correct credentials,
- **Then** they receive a valid session and are redirected to the dashboard.

(Each major feature should have similar AC covering happy path, validation failure, permission failure, error state, and edge case.)

30. QA & Testing Requirements

- **Unit tests:** Domain logic (stock calculations, discounts, state transitions).
- **Integration tests:** API endpoints, payment webhooks, WhatsApp message handling.
- **E2E tests:** Key journeys (onboarding, POS sale, WhatsApp order flow).
- **Security tests:** AuthZ checks, webhook signature verification, OWASP tests. ^[22]
- **Performance tests:** POS under load, large product catalogs.
- **Accessibility tests:** Keyboard nav, screen reader basics.
- **Failure/recovery tests:** Offline POS, webhook retries, network blips. ^[11] ^[13] ^[14]

High-risk areas: payments, stock integrity, multi-tenant isolation, offline sync.

31. AI Development Requirements

Requirements are written to be explicit for AI coding agents:

- Every entity has clear fields, relationships, and lifecycle rules.
- Every feature has actors, triggers, success/failure states, and audit needs.
- Permissions and org/branch scoping are explicit.
- State machines and business rules are enumerated.
- Edge cases and error handling are described.

AI agents should:

- Generate domain models matching entities.
- Implement RBAC checks on every write.
- Enforce idempotency on payments and sales.
- Implement offline queue + sync for POS. ^[13] ^[14] ^[11]

32. Assumptions

Confirmed / Established

- Product is Nigerian-focused, NGN, WhatsApp + local payments integrated. ^[3] ^[1] ^[7] ^[2]
- MVP scope: Inventory + POS + WhatsApp Orders + Payments. ^[9] ^[6] ^[2]
- Multi-tenant SaaS with org/branch model.

Strong Assumptions

- Primary payment provider: Paystack; secondary: Flutterwave/Moniepoint. ^[4] ^[20] ^[3]
- WhatsApp via BSP (WATI/Twilio/MessageBird or Nigerian BSP). ^[27] ^[26] ^[3]
- SMBs need offline-capable POS due to connectivity issues. ^[14] ^[11] ^[13]

Open Questions

- Exact pricing tiers and limits.
- Whether to support virtual accounts (Monnify) in MVP.
- Depth of NDPA tooling (DPO appointment threshold, self-serve data export UI timing). ^[18]

Risks

- Payment/WhatsApp API changes or downtime.
- SMB price sensitivity limiting ARPU. ^[16]
- Offline sync complexity underestimation. ^[30] ^[14]

33. Open Product Decisions

ID	Decision	Options	Impact	Info Needed
PD-01	MVP payment providers	Paystack only vs Paystack + Flutterwave	Dev effort vs merchant flexibility	Early merchant preferences, dev capacity
PD-02	WhatsApp BSP choice	Global (Twilio/MessageBird) vs Nigerian BSP	Cost, compliance, support	Pricing, NDPA DPA terms, reliability
PD-03	Free tier limits	Transaction count vs feature-limited	Conversion vs adoption	Willingness to pay, competitor benchmarks ^[2] ^[16]
PD-04	Offline conflict strategy	Last-write-wins vs explicit variance workflow	UX complexity vs data accuracy	Tolerance for manual resolution

34. Product Risks

- **Business:** Low willingness to pay; high churn if ROI not clear. ^[12] ^[16]
 - *Mitigation:* Tight onboarding, clear profit reports, pilot with 10–20 shops.
- **Technical:** Offline sync bugs causing stock mismatches. ^[11] ^[14]
 - *Mitigation:* Robust conflict rules, audit, manual adjustment workflows.
- **Security:** Webhook spoofing, tenant data leakage. ^[22]
 - *Mitigation:* HMAC verification, strict org scoping, pen tests.
- **Operational:** BSP/payment provider outages.
 - *Mitigation:* Multi-provider fallback, clear status comms.
- **Adoption:** SMBs resistant to change. ^[12]
 - *Mitigation:* WhatsApp-first UX, CSV import, hands-on support.

35. Competitive / Market Context

- **Market size:** Nigeria inventory software ~USD 75M (2025) → ~USD 168M (2031). ^[15] ^[1]
- **Alternatives:**
 - **Global ERPs (Odoo, Zoho, SAP):** Broad but heavy/expensive for SMBs. ^[5] ^[12]
 - **Local POS (InkeepX, Vendloop, Supply Smart):** Strong on POS/payments, weaker on deep inventory/ops. ^[31] ^[32] ^[9] ^[2]
 - **Payment-only (OPay, Moniepoint POS):** Great for payments, limited inventory control. ^[33] ^[34]
- **Orvio differentiation:** Unified inventory + POS + WhatsApp + payments, Nigeria-native UX/pricing, offline-first. ^[7] ^[11] ^[2]

36. Final Requirements Summary

- **Core capabilities:** Multi-tenant inventory, offline POS, WhatsApp orders, local payments, roles/permissions, basic reports. [\[3\]](#) [\[7\]](#) [\[9\]](#) [\[6\]](#) [\[2\]](#)
- **Core journeys:** Onboarding, inventory setup, POS sale, WhatsApp order → payment, subscription billing.
- **Core entities:** Org, Branch, Product, StockItem, StockMovement, SalesOrder, PaymentIntent, WhatsAppOrder, Subscription, AuditLog.
- **Core roles:** Owner, Manager, Cashier, Storekeeper, Accountant, Internal Admin.
- **Core business rules:** No negative stock, role-based discounts, idempotent payments, NDPA-aware data handling. [\[23\]](#) [\[21\]](#) [\[10\]](#) [\[18\]](#)
- **MVP scope:** As defined in Section 27.
- **Critical dependencies:** Auth → Org → Inventory → POS → WhatsApp → Payments.
- **Critical risks:** Offline sync integrity, payment/WhatsApp outages, price sensitivity. [\[12\]](#) [\[14\]](#) [\[16\]](#)
- **Open decisions:** Payment provider mix, BSP choice, free tier design, conflict strategy.
- **Success metrics:** Activation rate, 90-day retention, % orgs using WhatsApp, MRR growth.

37. Requirements Traceability Matrix (excerpt)

Requirement ID	Feature	User(s)	Business Goal	Dependency	Acceptance Criteria	Phase
INV-001	Create Product	Owner, Manager	Accurate catalog for stock control	ORG-002 (Branch)	AC-INV-001	MVP
INV-003	Stock Adjustment	Storekeeper, Manager	Reduce shrinkage, track variances	INV-001, INV-002	AC for adjustment flow	MVP
POS-001	Create Sale	Cashier, Manager	Enable fast in-store sales	INV-001, INV-002	AC-POS-001	MVP
WA-002	WhatsApp Order	System, Customer	Capture WhatsApp sales automatically	INV-001, PAY-001	AC-WA-001	MVP
PAY-002	Payment Webhook	System	Auto-confirm orders on payment	PAY-001, WA-002	Idempotent, verified	MVP
AUTH-001	Registration	New user	Acquire new orgs	None	AC-AUTH-001 (analogous)	MVP
ORG-001	Create Org	Owner	Multi-tenant foundation	AUTH-001	Org created, user owner	MVP

(Full matrix should cover all major requirements.)

This PRD is intended to serve as the single source of truth for design, architecture, development, QA, and AI coding agents building Orvio MVP and subsequent phases. [\[1\]](#) [\[3\]](#) [\[7\]](#) [\[13\]](#) [\[14\]](#) [\[6\]](#) [\[11\]](#) [\[2\]](#) [\[10\]](#)

1. <https://www.kenresearch.com/industry-reports/nigeria-inventory-management-software-market>
2. <https://www.inkeepx.com/best-pos-software-nigeria-comparison>
3. <https://www.go4whatsup.com/nigeria/>
4. <https://kudicompass.com/flutterwave-vs-paystack-vs-moniepoint-nigeria/>
5. <https://ramp.com/blog/what-is-odoo-erp>
6. <https://www.raspibtech.com/blog/inventory-management-software-nigeria>
7. <https://sabnode.com/blog/whatsapp-catalog-commerce-guide>
8. <https://startmessaging.com/ng/blog/whatsapp-social-commerce-nigeria/>
9. <https://vendloop.com/>
10. <https://bossbot.uk/blog/payment-links-ng>
11. <https://www.cleverence.com/articles/business-blogs/how-to-real-time-sync-for-retail-4827/>
12. <https://impactinternationaljournals.com/wp-content/uploads/2025/07/THE-EFFECT-OF-ENTERPRISE-RESOURCE-PLAN-NING-ERP-USAGE-ON-SME-PROFITABILITY-IN-SOUTHWESTERN-NIGERIA.pdf>
13. <https://www.linkedin.com/pulse/building-offline-pos-system-prateek-kaushik-5r1qf>
14. <https://contra.com/community/Z5fRds60-how-offline-first-pos-systems-keep-businesses>
15. <https://www.kenresearch.com/nigeria-inventory-management-software-market>
16. https://www.linkedin.com/posts/inkeepx_how-much-should-pos-software-cost-in-nigeria-activity-7491582604991770624-303R
17. <https://awajis.com/online-payment-gateways-in-nigeria/>
18. <https://www.bossbot.uk/blog/setup-guide-ng>
19. <https://whatsappbusiness.com/products/business-platform/>
20. <https://support.paystack.com/en/articles/2123458>
21. <https://paystack.com/docs/payments/webhooks/>
22. <https://www.kobosync.com/blog/2026-06-25-webhook-security-best-practices-nigeria>
23. <https://www.mctaba.com/learn/paystack/paystack-in-nigeria-complete-developer-guide>
24. <https://www.mctaba.com/learn/nigeria/integrate-paystack-nigeria>
25. <https://www.jumia.com.ng/point-of-sale-pos-equipment-d/>
26. <https://bossbot.uk/blog/manychat-alternativa-ng>
27. <https://succeedhqinnovations.com/buy-with-whatsapp-ecommerce-nigeria/>
28. <https://paystack.com/docs/>
29. <https://paystack.com/docs/payments/recurring-charges/>
30. <https://askbiz.co/academy/rural-retail-digital-infrastructure-pos>
31. <https://www.softwaresuggest.com/point-of-sale-pos-software/nigeria>
32. <https://www.supplysmart.co/pos-for-small-businesses-in-nigeria/>
33. <https://truehost.com.ng/top-10-best-pos-machines-in-nigeria/>
34. <https://digiconceptng.com/blog/best-pos-machines/>

35. <https://paystack.com/docs/payments/accept-payments/>

36. <https://paystack.com/docs/payments/payment-channels/>